DeepStream NV optical flow and motion direction analysis

DeepStream NV optical flow and motion direction analysis

1. Case goal and processing chain
2. Official application and environment
3. Run entry
4. Running the local video
5. Running with the cameras and the hardware limits
6. Observation and acceptance
7. Stopping and common problems
8. References

> Applicable environment: JetPack 7.2.1, Ubuntu 24.04, DeepStream 9.1, CUDA 13.2, TensorRT 10.16, Jetson Orin NX.

> This case is based on NVIDIA DeepStream `reference_apps/anomaly`. It demonstrates the TrafficCamNet object detection, the NVIDIA hardware optical flow and the motion direction analysis; it is not a general anomaly classification model that can directly judge any scene as "normal/abnormal".

1. Case goal and processing chain

This case shows the detection results and the motion information between adjacent video frames at the same time:

```
Input video/camera
       ↓
Decoding and nvstreammux
       ↓
TrafficCamNet object detection
       ↓
nvof hardware optical flow → dsdirection direction analysis
       ↓                        ↓
nvofvisual optical flow preview  detection/direction info preview
```

`nvof` computes the pixel motion vectors, `nvofvisual` draws them as a false-colour image, and `dsdirection` performs the direction analysis according to the motion information of the targets. The optical flow window shows the motion vectors, not an anomaly category or an anomaly probability; the specific business alarms still have to be combined with the scene rules and the application layer logic.

2. Official application and environment

Official reference application: https://github.com/NVIDIA/DeepStream/tree/main/src/apps/reference_apps/anomaly

Source code directory:

```
~/deepstream_reference_apps/anomaly/apps/deepstream-anomaly-detection-test
```

The course device already has the application, the TrafficCamNet model and the TensorRT engine, as well as the unified startup script. For the daily runs you do not need to download the models again, delete the engines or recompile the program. The engines are tied to the TensorRT/CUDA environment of the device; when restoring the environment you should check and build them on the target Jetson following the official instructions, and you must not directly copy engines that were generated on another device.

3. Run entry

The unified startup script is located at:

```
~/deepstream_tao_demo/
```

Input source	Command	Current output
Local video	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_optical_flow.sh`	Detection preview + optical flow false-colour dual window
USB camera	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_optical_flow_usb.sh`	Optical flow + detection when the PVA is available; stably degraded to the detection preview when there is no PVA
CSI camera	`cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_optical_flow_csi.sh`	Optical flow + detection when the PVA is available; stably degraded to the detection preview when there is no PVA

The entries above only preview and do not generate or overwrite an MP4. The desktop window needs the graphical desktop of the local Jetson or a VNC graphical session; a pure SSH session without a graphical display cannot show the preview.

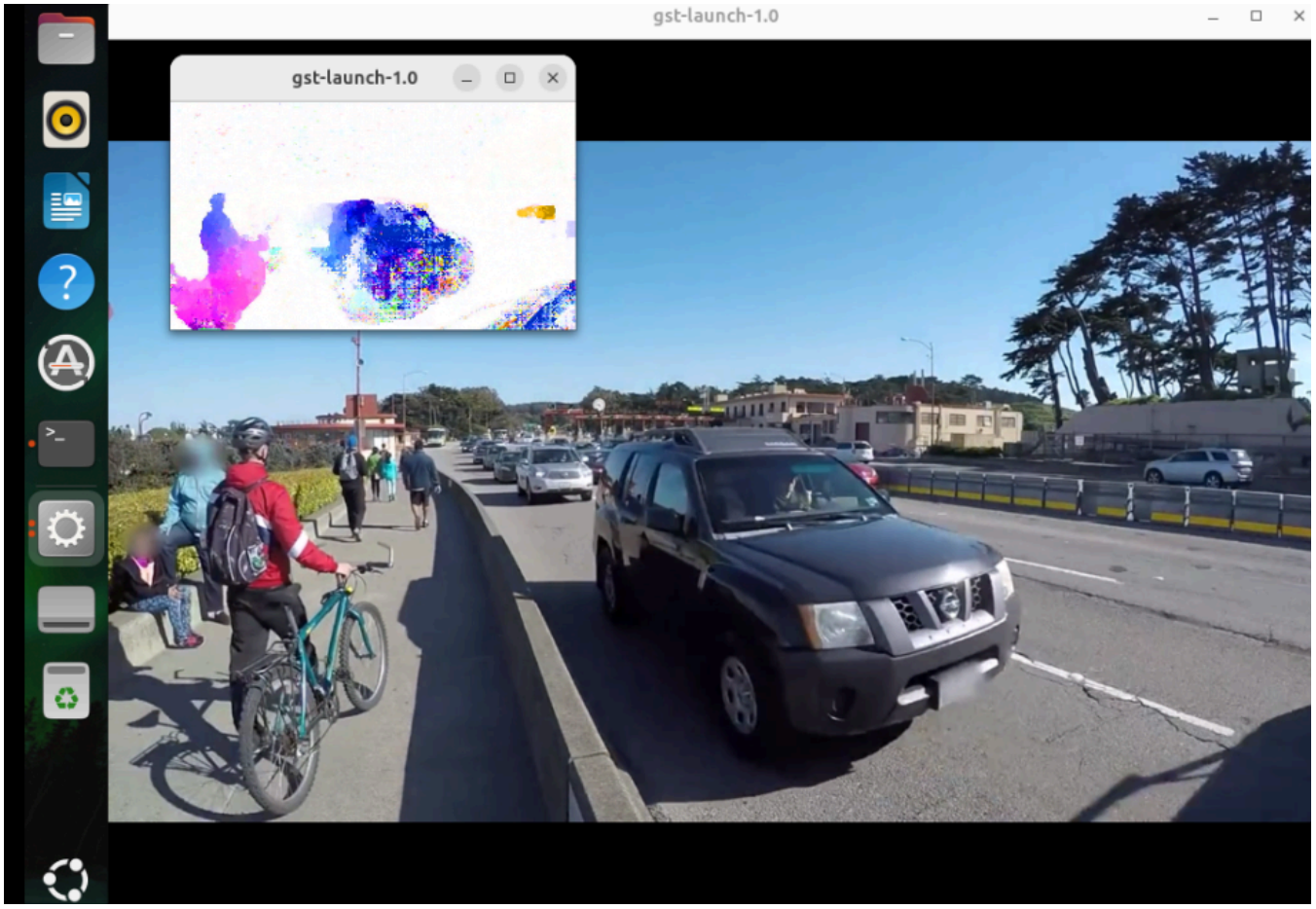
4. Running the local video

Run it from the local Jetson desktop or from a VNC terminal:

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_optical_flow.sh
```

The default input is a DeepStream sample video:

```
/opt/nvidia/deepstream/deepstream-9.1/samples/streams/sample_720p.h264
```



The TrafficCamNet detection window is displayed during the run. The script first checks `/dev/pva*`; only when it exists does it enable the `nvof` + `dsdirection` optical flow window.

To replace the test video:

```
./run_optical_flow.sh /home/jetson/Videos/test.mp4
```

The path must be an absolute path that exists on the Jetson.

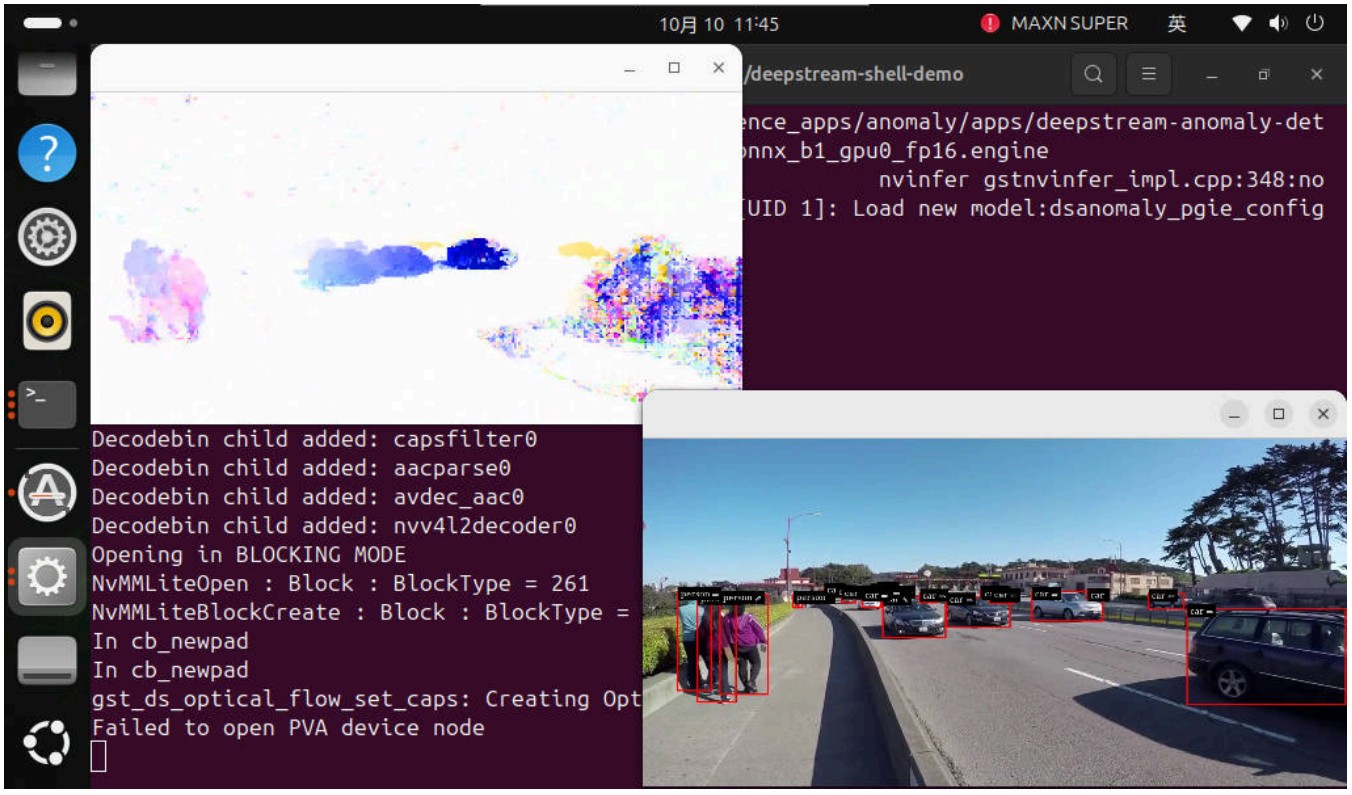
5. Running with the cameras and the hardware limits

USB camera:

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_optical_flow_usb.sh
```

CSI camera:

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_optical_flow_csi.sh
```



Before running, confirm that the camera is connected and is not occupied by another program. For the USB you can list the devices and the supported formats with the following commands:

```
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats-ext
```

6. Observation and acceptance

In the detection preview you should see the target boxes and the classes detected by TrafficCamNet. On a device where the hardware optical flow is enabled, the optical flow false-colour image should change continuously with the motion of the objects in the frame or when the camera is moved; it is normal for a static frame to produce few motion vectors.

The acceptance of a valid optical flow must satisfy all of the following at the same time:

1. There are no persistent PVA device open errors in the log.
2. A frame with motion produces an optical flow result that changes over time, instead of fixed noise.
3. The target boxes in the detection window change along with the input frame.

7. Stopping and common problems

Press `Ctrl+C` once in the terminal where you launched the command, wait for the program to exit and return to the Shell. Do not send interrupt signals repeatedly and do not use `kill -9`; the camera has to exit before another program can use it.

Symptom	How to handle it
`Failed to open PVA device node`	The current hardware/the device tree does not provide a usable PVA; the USB/CSI detection preview can still be used, but the hardware optical flow cannot be accepted.
`OF METADATA NOT FOUND` or the optical flow window is noise	It means that no valid optical flow metadata was obtained; the window itself does not mean that the optical flow succeeded. Check the PVA/driver state.
The USB reports that the device is busy or shows no image	Use `v4l2-ctl --list-devices` and `--list-formats-ext` to check the device node, the format and the occupancy.
The CSI reports that there is no camera	Check the CSI ribbon cable, the sensor, the `sensor-id` and the `nvargus-daemon` again.
There is only the window and no target boxes	Check whether the target is clearly visible and whether the lighting is sufficient, and confirm that the TrafficCamNet engine can be loaded normally.
There is no window after starting over SSH	Run it from the Jetson desktop/VNC session and check the `DISPLAY` and the X11 authorisation.
The USB/CSI optical flow window shows no person	The person has to move; only a continuous change produces a reaction.

8. References

- NVIDIA DeepStream Anomaly Detection: https://github.com/NVIDIA/DeepStream/tree/main/src/apps/reference_apps/anomaly
- NVIDIA DeepStream SDK: https://docs.nvidia.com/metropolis/deepstream/dev-guide/index.html